

ZNS SSD를 활용한 키-밸류 스토어의 성능 개선 연구



팀명: numpy

202055524 김의현

202055532 나도윤

202055545 박정민

지도교수: 안성용

제출일: 2025년 9월 19일

부산대학교

목차

1	서론	1
1.1	연구 배경 및 기존 문제점	1
1.2	연구 목표	1
2	연구 배경	3
2.1	ZNS SSD	3
2.2	LSM-Tree	4
2.3	Wisckey	5
2.4	RocksDB	6
2.4.1	Integrated BlobDB	6
2.4.2	Compaction	7
2.5	ZenFS	7
2.6	ConfZNS	7
3	연구 동기	8
3.1	현 BlobDB 벤치마크	8
3.1.1	실험 환경 및 설명	8
3.1.2	결과	9
3.2	원인 파악	10
3.2.1	최상위 레벨로의 Compaction 실패	10
3.2.2	Low Space Utilization	12
4	연구 내용	14
4.1	SIA: Sst file Isolated Allocation	14
4.2	SLSIA: Strict Lifetime-aware SIA	15
5	연구 결과 분석 및 평가	16

6 결론 및 향후 연구 방향	19
7 구성원 별 역할 및 개발 일정	20
7.1 연구일정	20
7.2 구성원 역할 분담	20
8 산업체 멘토 자문의견서 대응 내역	22
8.1 Lifetime Hint 관련	22
8.2 ZenFS의 파라미터 튜닝 관련	23
8.3 WAF 측정 관련	23
8.4 ‘문제 정의’, ‘과제 목표’ 관련	23

제1장 서론

1.1 연구 배경 및 기존 문제점

오늘날 많은 어플리케이션은 계산 중심이 아닌 데이터 중심적이다. 다양한 데이터를 다룰 일이 점점 증가하는 추세이며, 이에 따라 관계형 모델을 채택하지 않은 DB(Database)의 수요도 늘고 있다. 이러한 DB를 NoSQL이라 부르기도 하며, 키-밸류 스토어도 여기에 속한다. 특히 LSM-Tree(Log-Structured Merge Tree) 구조의 키-밸류 스토어를 많이 활용한다. 이러한 LSM-Tree는 쓰기 위주의 워크로드에 강하며, 랜덤 쓰기를 순차 쓰기로 전환하는 효과가 있다. SSD와 같이 활용할 시 여러 이점이 있어 자주 활용된다. 대표적인 키-밸류 스토어로 RocksDB [1], LevelDB[2]가 있다.

LSM-Tree는 상당한 양의 read/write amplification이 발생한다는 단점이 있다. 특히 write amplification은 SSD의 수명에 직결되므로, 이러한 write amplification을 줄이는 것이 중요하다. 전통적으로는 Compaction 전략을 개선하는 방향의 연구가 많이 이루어졌으며, 밸류를 LSM-Tree에서 별도로 분리하여 write amplification을 줄이는 접근 또한 힘을 얻고 있다. Wiskey[3], Badger[4] 등이 밸류를 LSM-Tree에서 분리하는 접근을 취하고 있으며, RocksDB도 integrated BlobDB 옵션을 활성화하면(이하 BlobDB) 밸류를 별도의 파일에 저장한다 [5].

ZenFS는 ZNS SSD(Zoned Namespace SSD)를 지원하는 RocksDB plugin이다. ZenFS는 파일의 lifetime을 잘 고려하여 각 Zone에 배치하는 것으로 GC(Garbage Collection) 효율을 높이고 SSD의 수명을 개선하는데, 기존 파일시스템 대비 상당한 성능 향상이 있다 [6].

문제는 ZenFS가 BlobDB를 잘 지원하지 못하는 것이다. BlobDB와 ZenFS를 모두 활성화하여 벤치마크를 진행하였을 때, Compaction failure가 반복적으로 발생하였으며, space utilization이 낮은 것을 확인하였다.

1.2 연구 목표

본 연구는 BlobDB와 ZenFS를 동시에 운용할 경우 발생하는 문제를 해결하고자 한다. 먼저, 반복적인 Compaction failure를 해결한다. 현재 ZenFS의 lifetime hint 기반 zone al-

location 정책은 BlobDB의 Compaction 패턴과 부조화를 이루어, empty zone 고갈로 인한 할당 실패가 빈번하게 발생한다. 이를 해결하기 위하여, 새로운 sst file 할당 전략인 SIA(SST file Isolated Allocation)을 제안한다. 다음으로, ZenFS의 낮은 space utilization 문제를 개선한다. 기존 ZenFS는 서로 다른 lifetime을 갖는 파일을 동일한 zone에 배치하여 zone reclaim이 잘 되지 않고, space utilization이 떨어진다. 이를 해결하기 위해, blob file을 lifetime hint 별로 zone에 배치하는 SLSIA(Strict Lifetime-aware SIA)를 제안한다. 해당 연구 결과로 반복적인 Compaction failure를 없앴으며, space utilization 또한 72.9%에서 96.03%로, 대폭 개선하였다.

제2장 연구 배경

2.1 ZNS SSD

기존 SSD는 블록 인터페이스를 바탕으로 데이터를 자유롭게 읽고 쓰도록 설계되어 있다. 이는 하드디스크에서 파생된 모델로, 논리적 블록 주소(Logical Block Address, 이하 LBA)를 기존처럼 다루기 때문에 호환성 측면에서 유리하다. 그러나 플래시 메모리는 특성상 덮어쓰기가 불가능하고, 삭제는 블록 단위로만 가능하다. 따라서 덮어쓰기가 자유로이 가능한 하드디스크와 근본적인 차이가 있다. 이를 기존 인터페이스와 호환시키기 위해, SSD 내부에 FTL(Flash Translation Layer) 계층이 추가로 필요하다. FTL은 out-of-place write DRAM과 over-provisioning 영역이 요구되며 성능 저하를 유발하는 garbage collection이 발생한다. 이러한 구조적 비효율을 통칭 ‘Block Interface Tax’라 부른다 [6].

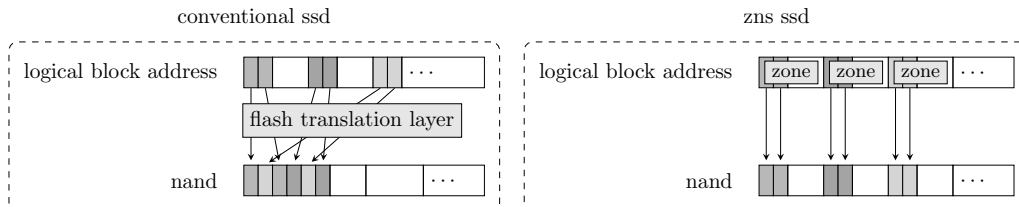


그림 2.1: 기존 SSD와 ZNS 비교

이를 해결하기 위해 제안된 ZNS는 LBA 공간을 여러 개의 Zone으로 나누고 각 Zone에 대해 순차적 쓰기만 허용하는 새로운 인터페이스이다. 그림 2.1은 기존 SSD와 ZNS SSD의 내부 구조를 비교한 것이다. 기존 SSD에서는 데이터가 완전히 FTL에 의해 배치되며, 완전한 블랙박스에 가깝다. 그러나 ZNS SSD에서는 Zone에 따라 데이터가 배치되므로, 개별 파일 시스템이나 어플리케이션이 데이터 배치를 지정할 수 있다. 따라서 종전에는 불가능했던, 데이터 배치 관련 최적화가 가능해지며, 이를 통해 쓰기 증폭(write amplification)을 최소화하며 SSD의 용량과 수명을 향상시킬 수 있다.

ZNS의 쓰기 제약과 Zone 동작 원리

ZNS SSD는 각 zone에 대해 쓰기 순서를 엄격히 제한한다. 하나의 Zone은 처음에 EMPTY 상태로 시작하며, 쓰기가 시작되면 OPEN 상태로 전이된다. 이후 쓰기가 끝나면 zone은 CLOSED 또는 FULL 상태가 되며, 추가 쓰기를 위해서는 반드시 reset 명령을 통해 초기화되어야 한다. 이때 호스트는 각 Zone에 대해 현재 쓰기 가능한 위치를 나타내는 write pointer를 관리해야 하며 만약 sequential write 제약을 위반하면 에러를 반환한다.

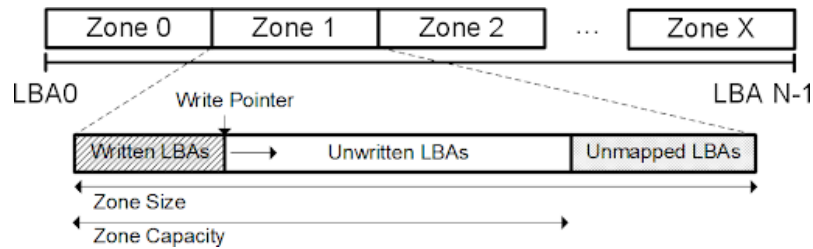


그림 2.2: Zone의 구성 방식. (출처: [6])

그림 2.2는 ZNS SSD에서 zone이 어떻게 구성되는지를 보여준다. 각 zone은 일정한 크기의 논리 주소 공간을 가진다. 각 논리 주소 공간은 쓰여진 영역(Written LBAs)과 쓰이지 않은 영역(Unwritten LBAs)으로 구분되고 write가 발생하면 write pointer가 점진적으로 증가한다. 이러한 구조는 호스트가 명시적으로 데이터를 배치하고 블록 지우기 단위를 직접 고려하게끔 하여, 기존 SSD에서 SSD 내부에 숨겨져 있던 관리 작업을 호스트 측에서 명확하게 제어할 수 있도록 만든다.

2.2 LSM-Tree

전통적인 디스크 기반 인덱스 구조인 B-Tree는 조회 성능에는 강점을 보이지만 삽입 및 갱신이 빈번한 워크로드에서는 성능 저하를 유발한다. 특히 트랜잭션 시스템에서 로그 파일이나 History 테이블처럼 지속적으로 새로운 데이터가 추가되는 경우 B-Tree 인덱스를 유지하기 위한 실시간 갱신 비용은 전체 I/O 부하를 사실상 두 배로 증가시켜 시스템 비용을 50% 이상 높일 수 있다 [7].

이러한 문제를 해결하기 위해 제안된 Log-Structured Merge-tree (LSM-Tree)는 변경 사항을 메모리에 우선 저장한 후 배치(batch) 방식으로 디스크에 순차적으로 반영하는 구조를 채택한다. 이 과정에서 병합(compaction)은 정렬 병합과 유사한 방식으로 수행되며 디스크 탐색 비용을 최소화하면서도 인덱스 접근 가능성을 유지한다. 그 결과, LSM-Tree는 삽입과 삭제가 많은 워크로드에서는 기존의 B-Tree에 비해 낮은 비용으로 인덱스를 유지할 수 있는 이점이 있다.

LSM-Tree는 메모리 상에 유지되는 C_0 Tree와 디스크 상의 다단계 구조(C_1-C_k)로 구성된다. 새로운 데이터는 우선 메모리의 C_0 Tree에 삽입되고, 그 후 일정 크기에 도달하면 디스크로 Flush 된다. 이후 여러 SST(Sorted String Table) File을 병합하면서 정렬 상태를 유지한다. 그림 2.4는 C_0 Tree가 디스크의 C_1 Tree와 병합되는 과정을 보여준다.

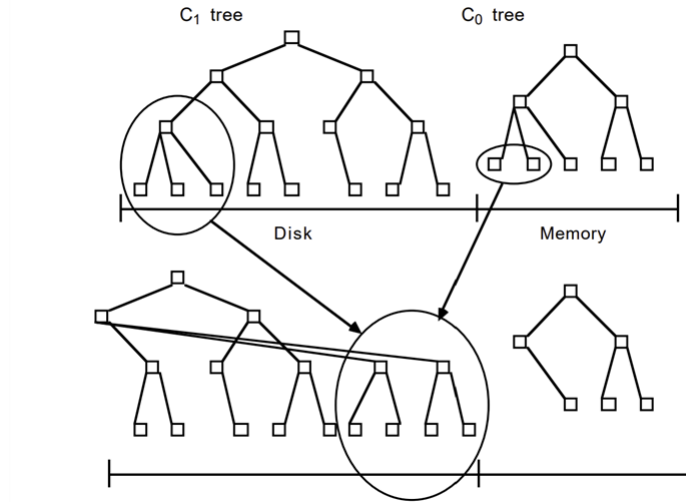


그림 2.3: C_0 Tree와 C_1 Tree가 병합되는 과정 (출처: [7])

이러한 구조는 단순히 두 계층에 그치지 않고 다단계 구조로 확장된다. 2.3은 C_0 (메모리)와 C_1 C_k (디스크)의 관계를 보여준다. 각 레벨은 상위 레벨에서 데이터를 병합해 저장한다. 이를 통해 hot-cold 데이터 분리가 가능해지고 결과적으로 hot 데이터는 상위 레벨에, cold 데이터는 하위 레벨에 위치하게 된다 [6].

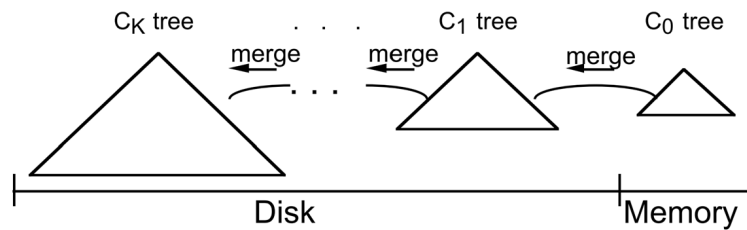


그림 2.4: 멀티 레벨 LSM-Tree의 구조 (출처: [7])

2.3 Wisckey

Wisckey는 LSM-Tree의 크기를 줄여 write amplificaion을 줄인 연구이다 [3]. 먼저 value가 상당히 크다고 가정해보자. 이때 value를 별도의 로그에 저장하고, LSM-Tree에는 밸류의 위치만을 저장하면 트리의 크기가 상당히 줄어든다. 더해서 Compaction 시 value를 이동하지 않기 때문에 데이터 이동을 많이 줄일 수 있게 된다. 즉, write amplification이 줄어든다.

대신, value가 저장된 로그를 별도로 관리하여야 한다는 문제가 생긴다. 기존의 LSM-Tree는 롤 Compaction 과정에서 obsolete된 key-value 쌍이 자연스럽게 삭제되나, Wiskcey는 Compaction 중 value를 옮기지 않기 때문이다. Wiskey는 별도의 value GC를 도입해서 이 문제를 해결하였다.

2.4 RocksDB

RocksDB는 FaceBook에서 개발한, LSM-Tree 구조의 키-밸류 스토어이다. 이번 절에서는 RocksDB, 특히 Integrated BlobDB 옵션을 활성화하였을 경우의 RocksDB를 다루고자 한다.

2.4.1 Integrated BlobDB

BlobDB는 Wiskey와 유사하게, value를 별도의 파일(blob file)로 분리하는 기능이다. Integrated BlobDB 옵션을 활성화한 RocksDB를 BlobDB라고 칭하기로 한다. 본 절에서는 BlobDB 위주의 내용을 간단히 다루기로 한다.

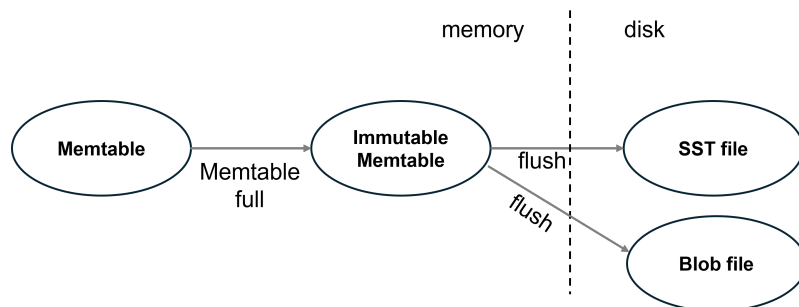


그림 2.5: BlobDB Flush

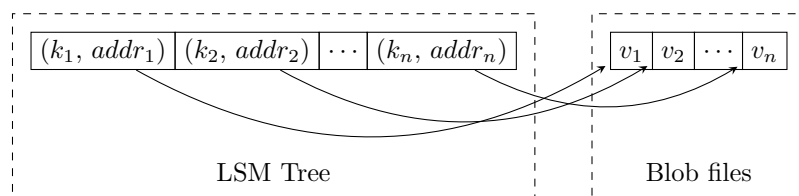


그림 2.6: BlobDB의 value separation

BlobDB는 in-memory 상에서는 vaule separation을 하지 않는다. 대신 Memtable이 Flush 될 때, sst file과 blob file로 분리한다(그림 2.5 참조). Flush 된 후에는 그림 2.6처럼 value를 sst file과 분리하여 저장한다.

2.4.2 Compaction

RocksDB는 다양한 Compaction scheme을 지원한다 [8]. 그러나 BlobDB 사용 시에는 Leveled Compaction을 사용할 것을 권하고 있으며, BlobDB의 특정 기능은 Leveled Compaction하에서만 동작한다 [9]. 이에 본 연구에서는 Leveled Compaction에 초점을 맞추기로 한다. 이후에 얘기하는 모든 Comaction은 별다른 언급이 없다면 Leveled Compaction을 의미한다.

독립적인 value GC를 사용하는 Wiskey와 달리, BlobDB는 Compaction 도중 obsolete 된 value를 처리한다. Compaction 도중 보이는 value가 GC 대상인 파일에 속한다면, 그리고 해당 value가 valid하다면 새로운 blob file로 옮긴다. 그렇지 않은 경우 value를 이동시키지 않는다. 매 Compaction 마다 정해진 비율의 blob file이 GC 대상이 되며(기본 설정은 전체의 25%), 단순히 오래된 순서대로 GC 대상을 고른다.

2.5 ZenFS

ZenFS는 Weston Digital에서 만든 RocksDB plugin으로, ZNS를 지원한다 [6]. RocksDB filesystem 인터페이스 하에 구현된 단순한 user level filesystem이다.

RocksDB는 LSM-Tree를 사용하는만큼 대부분의 I/O는 sequential I/O이다. 이 점이 sequential write 제약이 걸린 ZNS SSD와 자연스럽게 맞아떨어지며, ZenFS는 이를 잘 활용한다. 복잡한 추상화 계층 없이, RocksDB가 생성하는 파일들을 ZNS SSD의 Zone에 직접 매핑한다 [6]. LSM-Tree의 구조적 특성과 ZNS의 sequential write 제약이 자연스럽게 결합된 것이다.

2.6 ConfZNS

ConfZNS는 이번 과제에서 사용한 가상 머신이다. ConfZNS는 FEMU를 포크하여 개발된 플래시 에뮬레이터로, FEMU에 비해 ZNS 관련 지원을 강화하였다. FEMU는 QEMU의 포크이므로, 결국 QEMU의 포크이기도 하다.

FEMU는 기존의 에뮬레이터보다 정확하게 I/O latency를 모사한다. 기존 에뮬레이터에서 병목이 될만한 요소를 상당수 제거하여, I/O thread가 늘어도 비교적 안정적으로 latency를 유지한다 [10]. ConfZNS는 여기에 더해 Zone의 internal parallelism을 설정할 수 있으며, 더 정확한 타이밍 모델을 제공한다 [11].

제3장 연구 동기

본 장은 연구의 주된 동기가 된 BlobDB와 ZenFS간의 문제점을 다루고자 한다. 특정 시점부터 BlobDB의 Compaction이 반복적으로 실패하는 문제와 space utilization이 떨어지는 문제가 있었다. 각 현상이 잘 드러나는 실험을 제시하고, 그 원인을 다루고자 한다.

3.1 현 BlobDB 벤치마크

3.1.1 실험 환경 및 설명

표 3.1: 실험 환경

emulator	ConfZNS
number of zones	160
zone size	256MB
total ssd size	40GB
(guest) kernel version	6.1.0-37

표 3.2: BlobDB 설정

sst file size	32KB
blob file size	512MB
write buffer size	256MB
level multiplier	4
level base size	128KB

표 3.3: db_bench 설정

workload	overwrite
key size	16B
value size	128KB
entry	225047(약 27.5GB)

표 3.1과 같은 환경에서, 표 3.2와 같이 BlobDB를 설정하고, 표 3.3의 설정대로 설정하여

실험하였다. 벤치마크에 사용한 프로그램은 db_bench로, RocksDB의 내장 벤치마크 도구이다.

3.1.2 결과

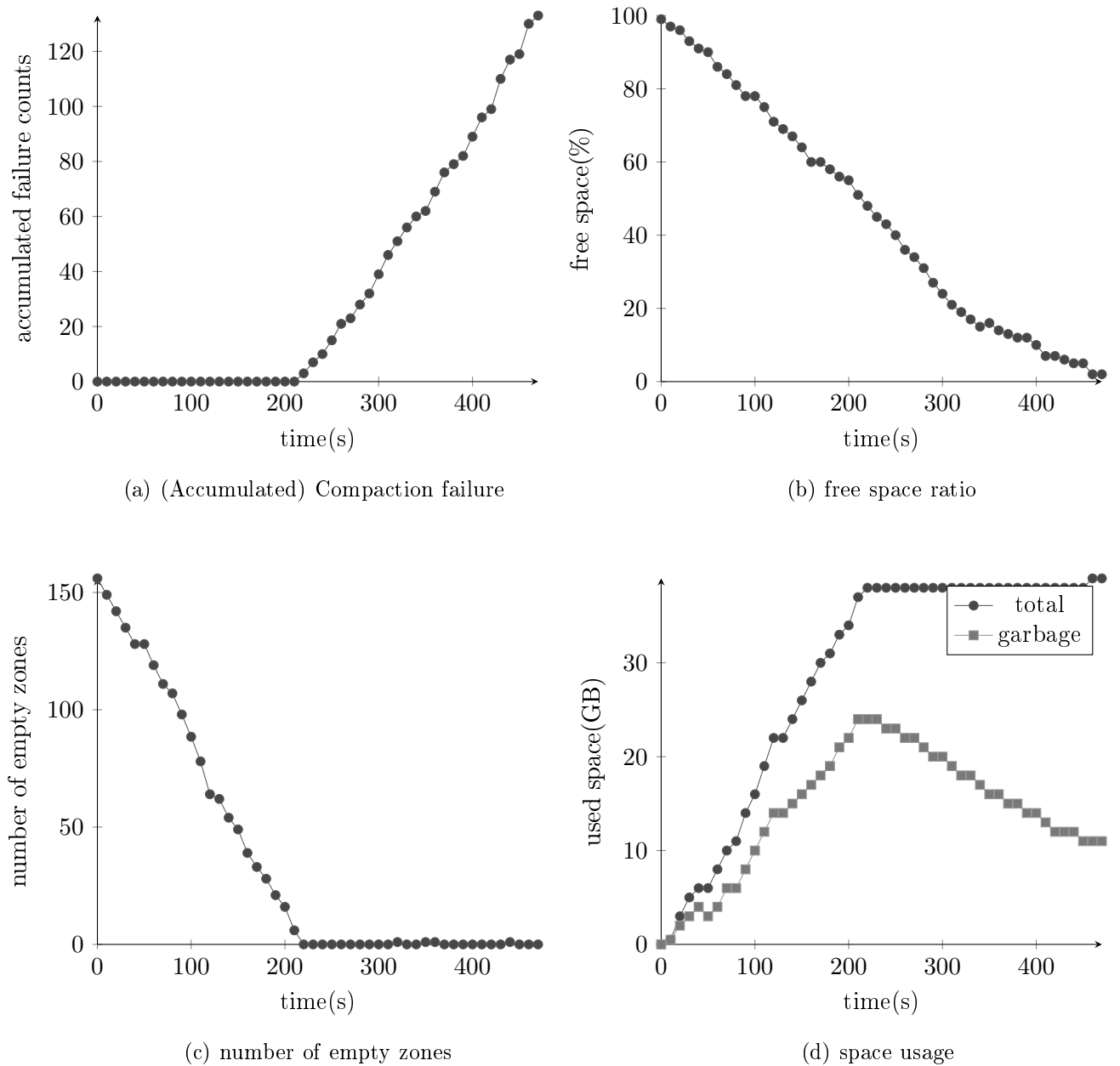


그림 3.1: 시간별 시스템 변화 추이

첫 번째로, 상위 레벨로의 Compaction이 반복적으로 실패하는 것을 식별하였다. 그림 3.1a는 벤치마크 도중 발생한 BlobDB 단의 에러의 수를 누적하여 나타낸 것이다. 특정 시점부터 점진적으로 증가하는 것을 확인할 수 있다. 로그 확인 결과, 최상위 레벨로의 Compaction이

반복적으로 실패하는 것을 확인할 수 있었다.

그러나, 그림 3.1b를 보면 알 수 있듯, Compaction failure가 발생한 시점은 실제로 free space가 상당히 남아있는 시점이니만큼, 이는 이상한 동작이다. 그림 3.1c와 같이 보면 더 명확해진다. 그림 3.1c는 empty zone의 수를 나타낸 그래프인데, empty zone 대부분을 소진한 시점부터(200s 부근) Compaction failure가 발생하는 것을 볼 수 있다.

두 번째 문제는 space utilization이 떨어지는 것이다. 해당 워크로드 종료 시점의 space utilization은 72.9% 정도였으며, 그림 3.1d를 보면, ZenFS의 space 사용량 패턴이 특이한 것을 확인할 수 있다. 200s 시점까지는 garbage space와 total used space가 선형적으로 증가하다가 정점에 도달한다. 그 이후부터는 total used space는 거의 일정한 반면 garbage space만이 감소하는 패턴을 보인다. garbage space가 reclaim 되자마자 재사용된 것이라고 볼 수 있다. 그리고

사실 워크로드 종료 시점의 space utilization이 다소 낮고, 다소 특이한 패턴으로 공간을 사용하더라도, reclaim이 잘 된다면 큰 문제가 되지 않을 것이다. 그래서 그 점을 검증해보기 위해 추가적으로 실험을 해 보았다. 현재 워크로드가 종료되었을 시점에 garbage data가 27%(약 10GB) 가량 있으므로, 현재 워크로드보다 2~3GB 정도로 추가로 write를 요청하더라도 수행할 수 있을 것이라 기대하였으나, 실험해본 결과는 그렇지 않았다. 전부 워크로드 수행에 실패하였다. 전체 디스크의 25%가량이 garbage임에도 reclaim이 안 된다는 뜻이니만큼, 분명한 문제라고 생각한다.

$$\frac{(\text{user written data}) + (\text{GC written data})}{(\text{user written data})} \quad (3.1)$$

마지막으로 특기할 점은, 모든 워크로드에서 ZenFS단의 write amplification은 1.03 정도로 상당히 낮게 측정되었다는 것이다. 식 3.1과 같이 write amplification을 계산하였는데, 이는 GC에서 데이터 이동이 거의 없었다고 볼 수 있다.

3.2 원인 파악

3.2.1 최상위 레벨로의 Compaction 실패

Compaction이 실패한 시점의 로그를 보면, ZenFS 단에서 반복적으로 zone allocation이 실패한다. 그에 따라 BlobDB단에서 Compaction이 실패하였다는 점, 그리고 해당 현상이 특정 시점부터 반복된다는 점을 확인할 수 있다. 해당 현상을 이해하기 위해서는 먼저, ZenFS의 zone allocation 방식과, BlobDB에서 file에 lifetime hint를 부여하는 방식을 이해하여야 한다.

알고리즘 3.1: (단순화한) ZenFS의 Zone 할당

```

입력: none empty zone의 리스트, empty zone의 리스트
출력: allocated zone
best_diff  $\leftarrow \infty$ 
best_zone  $\leftarrow \text{null}$ 
for zone  $\in$  none_empty_zones do
    diff  $\leftarrow$  (zone.lifetime - file_lifetime)
    if  $0 < \text{diff} \leq \text{best\_diff}$  then
        best_diff  $\leftarrow$  diff
        best_zone  $\leftarrow$  zone
if best_zone = null then
    best_zone  $\leftarrow$  AllocateEmptyZone()
    best_zone.lifetime  $\leftarrow$  file_lifetime

```

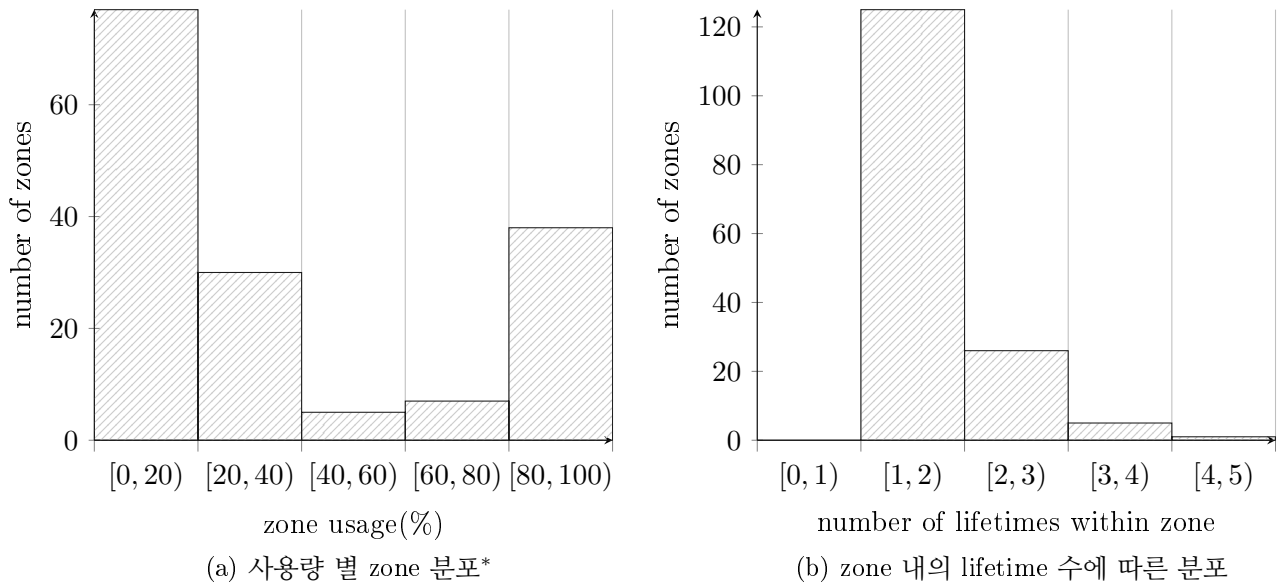
알고리즘 3.1은 ZenFS의 zone 할당 방식을 나타낸 것이다. 먼저, non-empty zone에 할당을 시도한다. lifetime hint를 바탕으로, file의 lifetime보다 긴 라이프타임 힌트를 갖는 zone을 찾아 할당한다. 실패할 경우, 새 zone을 할당하고, 해당 zone의 라이프타임을 할당한 file의 라이프타임으로 지정한다. 보다시피 file의 lifetime hint에 크게 의존한다.

이제 BlobDB의 lifetime hint 부여 방식을 보자. BlobDB는 level을 바탕으로 lifetime hint를 결정하며, 레벨이 높을수록 짧은 lifetime hint를, 낮을수록 긴 lifetime hint를 부여한다. 한 가지 특기할 점은, 개별 파일의 lifetime hint를 매 번 계산하는 것이 아니라는 것이다. 대신 매 Flush/Compaction마다 output이 될 level을 바탕으로 한 번만 계산한 뒤, 해당 lifetime hint를 공유한다. 달리 말하면 특정 Flush/Compaction의 결과물로 나온 sst file/blob file들은 모두 같은 lifetime hint를 가진다.

앞서 설명한 내용을 종합하면 어째서 최상위 level Compaction에서 반복적으로 오류가 나는지 이해할 수 있다. 최상위 level로의 Compaction이 일어난 경우, 가장 긴 lifetime hint를 가지기 때문에, 기존의 zone에는 할당될 수 없다. 즉, 무조건 새로운 zone을 할당받아야 하는데, 앞서 그림 3.1c에서 보았듯이, BlobDB는 empty zone을 아주 빠르게 소모한다. 따라서 할당 실패가 일어날 수 밖에 없다.

더해서, BlobDB에서 Compaction 실패는 복구 가능한 에러로 간주된다. Compaction이 실패하더라도 데이터 일관성에는 문제가 없기 때문이다. 실제로 free space는 꽤 남아있고, 다른 lifetime hint를 갖는 파일은 할당할 수 있기 때문에 워크로드를 수행할 수 있다. 그러나 무의미한 Compaction을 반복적으로 시도하고 복구하는것은 절대 바람직한 현상이 아니다.

정확한 원인을 파악하려면 failure가 일어난 시점에서의 zone 상태를 알 필요가 있다. 그림 3.2는 첫번째 Compaction 실패 시점에서의 zone의 상태를 시각화한 것이다. 그림 3.2a를 보면, 전체 용량의 20% 미만을 사용하는 zone이 상당히 많은 것을 확인할 수 있다. 그림



*empty zone은 집계하지 않음.

그림 3.2: 첫번째 Compaction 실패 시점의 Zone 상태

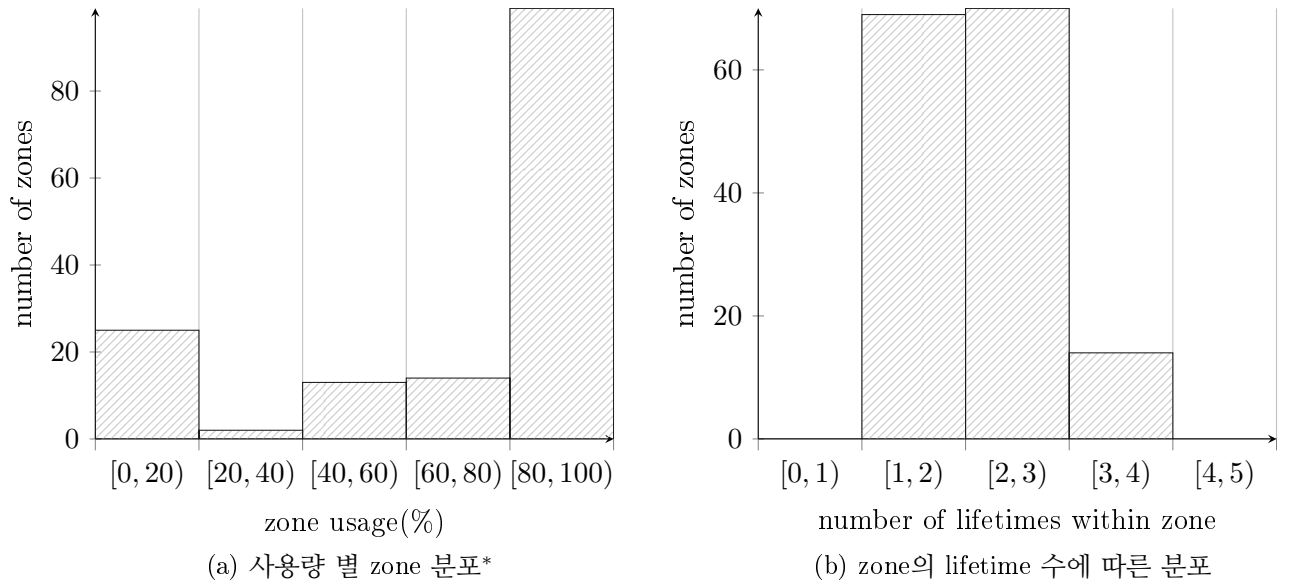
3.2b를 보면, 대부분의 zone이 단일한 lifetime hint를 가지는 것을 확인할 수 있다. 실제로 확인해보았을 때도 작은 크기의 파일 하나만이 할당된 zone이 많았으며, 상당수가 sst file이었다.

3.2.2 Low Space Utilization

개선 방향을 파악하기 앞서, ZenFS단에서 space reclaim이 어떻게 일어나는지 보자. free space를 기준으로, free space가 20% 되기 전까지는 사용하지 않는 zone(invalid data만이 있는 zone)을 reset하여 space를 reclaim한다. free space가 20% 아래로 떨어지는 경우, 자체적으로 GC를 수행한다. GC thread에서 주기적으로 utilization이 낮은 zone의 valid data를 옮기고 난 뒤 해당 zone을 reset한다. 따라서 zone reset이 잘 되도록 데이터 배치를 개선하거나, 아니면 GC를 더 공격적으로 수행하도록 하여야 한다.

그림 3.3b은 zone에 할당된 파일의 lifetime hint의 수를 바탕으로 히스토그램을 그린 것이다. 보다시피 서로 다른 lifetime hint를 갖는 파일이 섞여있는 zone이 상당히 많다. 그런데 하나의 zone에 서로 다른 lifetime hint를 가지는 파일이 뒤섞이는 것은 바람직하지 않다. 서로 다른 lifetime을 갖는 파일이 섞여 reclaim을 방해하기 때문이다.

더하여, ZenFS는 blob file과 sst file을 구분하지 않고 할당하는데, 문제의 소지가 있다. 그림 에서 볼 수 있듯이, sst file은 매 Compaction마다 입력으로 들어가는 sst file이 삭제된다. 그러나 blob file은 GC가 작동하는 경우에만 새로 생성/삭제되므로, 같은 lifetime hint 기준



*empty zone은 집계하지 않음.

그림 3.3: 워크로드 종료 시점의 zone 상태

blob file은 sst file보다 lifetime이 긴 편이다. 따라서 file-aware한 처리가 필요한데, ZenFS는 단순히 lifetime hint만 고려하고 할당한다.

제4장 연구 내용

본 장에서는 3장에서 찾은 문제의 해결책으로, sst file과 blob file 각각의 개선된 할당 전략을 제시한다.

4.1 SIA: Sst file Isolated Allocation

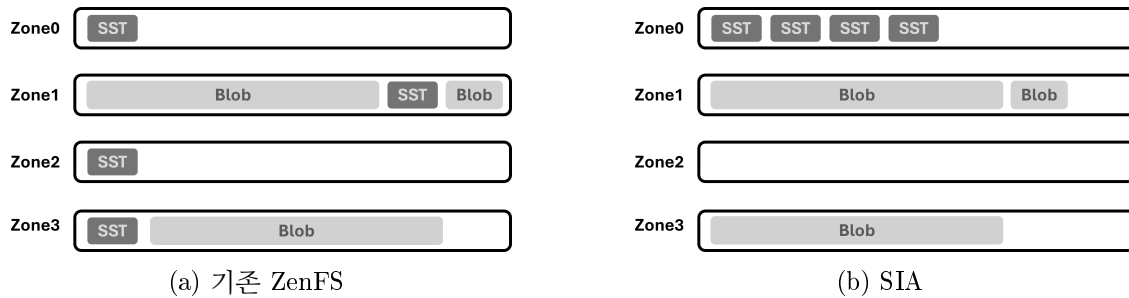


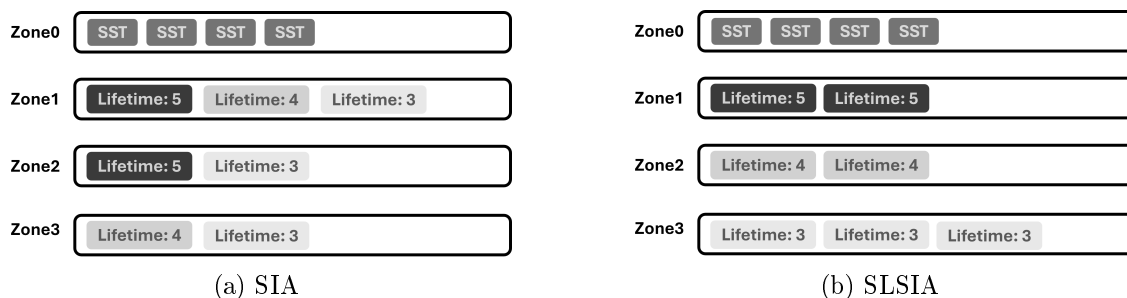
그림 4.1: 기존 할당 정책과 SIA의 비교

본 절에서는 Compaction failure 문제를 해결하기 위한 새로운 allocation scheme, SIA (Sst-Isolated Allocation)를 제시한다. 어찌하여 SIA가 Compaction failure 문제를 해결할 수 있을지, 그리고 그 밖의 예상되는 장점 또한 간략히 다룰 것이다.

기존 ZenFS는 그림 4.1a와 같이, sst file (혹은 blob file이) zone을 단독으로 차지하는 경우가 종종 생긴다. 더 짧은 lifetime hint를 가진 파일은 해당 zone에 할당될 수 있으므로, 일견 큰 문제가 되지 않는 것처럼 보일 수 있다. 그러나 3.2.1 절에서 보았듯, ZenFS는 empty zone을 공격적으로 소모한다. 그리고 최상위 level로의 Compaction은 무조건 empty zone을 요구하기 때문에 할당 실패가 빈번하게 일어나게 된다. 따라서 저러한 zone을 줄일 필요가 있다.

SIA는 그림 4.1b와 같이 sst file을 가능한 한 동일한 zone에 할당한다. sst file이 한 zone에 묶이게 되므로, empty zone 소모 속도가 상당히 줄어들 것을 기대할 수 있다. 또한 BlobDB의 특성상 sst file의 크기는 blob file에 비해 상당히 작기 때문에, sst file을 lifetime hint를 무시하고 할당해서 생기는 비효율이 그리 크지 않을 것이라 추측하였다.

4.2 SLSIA: Strict Lifetime-aware SIA



(a) SIA

(b) SLSIA

그림 4.2: SIA와 SLSIA간의 비교

본 절에서는 space utilization을 개선하기 위한 할당 정책, SLSIA(Strict Lifetime-aware SIA)를 제시한다. 그리고 어찌하여 해당 정책이 space utilization을 개선하는지 다루고자 한다.

먼저 SIA의 한계를 보도록 하자. SIA의 경우, 하나의 zone에 서로 다른 lifetime hint를 갖는 blob file들이 섞인다(그림 4.2a 참조). 문제는 blob file의 lifetime hint는 blob file의 lifetime을 잘 반영하기 때문에, 결론적으로 lifetime이 다른 데이터가 섞이는 결과를 초래하게 된다. 따라서 zone reclaim의 효율이 떨어지게 된다.

아울러, ZenFS의 보수적인 GC 정책이 문제를 가중한다. 다양한 환경으로 실험하여보아도, ZenFS 단에서는 write amplification이 별로 발생하지 않는 것을 관찰하였다. 달리 말하면, 대부분의 상황에서 단순히 garbage data가 가득찬 zone을 reset하기만 한다고 볼 수 있다. 따라서 더더욱 정교한 데이터 배치가 요구된다.

이에 SLSIA는 그림 4.2b와 같이, blob file을 lifetime hint별로 zone에 배치한다. sst file과 blob file을 분리함과 동시에, 같은 lifetime을 갖는 blob file만을 같은 zone에 배치하여 zone reclaim이 최대한 잘 되도록 유도한 것이다.

제5장 연구 결과 분석 및 평가

본 장에서는 SIA와 SLSIA 각각이 실제 의도대로 동작하는지 검증하고자 한다. 3.1.2 절과 동일하게 환경을 구성하여, 각 전략별로 실험을 진행하였다.

먼저 특기할 점은 SIA, SLSIA 모두에서 그림 5.1a와 같이, Compaction failure가 완전히 사라졌다는 점이다. 그림 5.1c에서 확인할 수 있듯이, 둘 모두 원래 전략에 비하면 zone 소모 속도가 줄어들었기 때문이다. 따라서 Compaction failure 개선은 소기의 목적을 달성했다고 할 수 있을 것이다.

policy	total used(GB)	garbage(GB)	space utilization
vanilla	39.5	10.7	72.9%
SIA	38.25	9.05	76.33%
SLSIA	30.27	1.2	96.03%

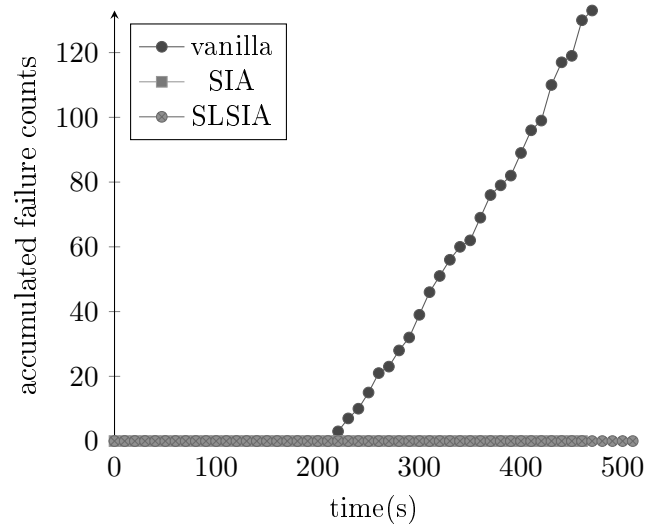
표 5.1: 워크로드 종료 시점의 space utilization

$$\text{space utilization} = \frac{\text{total used space} - \text{garbage space}}{\text{total used space}} \quad (5.1)$$

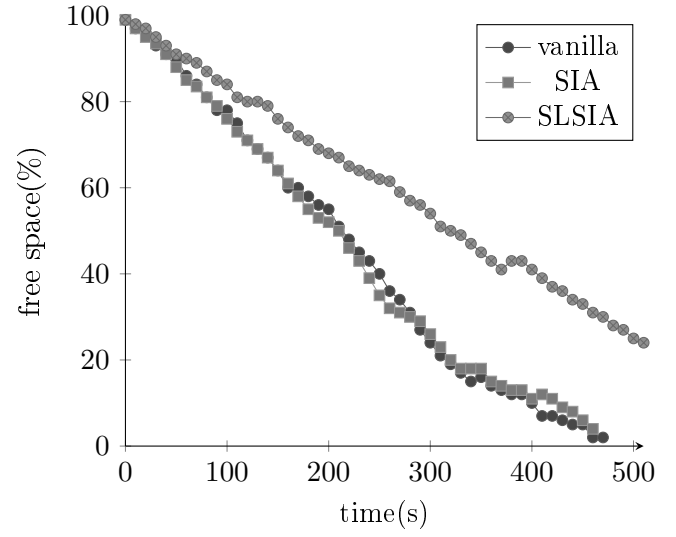
이제 space utilization을 보자. 그림 5.1b를 보면, SIA와 원래 ZenFS의 정책은 그리 큰 차이를 보이지 않는 것을 볼 수 있다. 반면에 SLSIA의 경우, 전반적으로 free space를 높게 유지하는 것을 볼 수 있다. 이는 워크로드 종료 시점의 zone 상태를 확인해보면 더욱 분명해진다. 그림 5.2는 SLSIA의, 워크로드 종료 시점의 zone의 상태를 나타낸 것이다. 대부분의 zone의 utilization이 아주 높은 것을 확인할 수 있다.

마지막으로 표 5.1은 워크로드 종료 시점의 space utilization을 나타낸 것이다. 식 5.1과 같이 계산하였으며, SLSIA가 96.03%로, 압도적으로 높은 것을 볼 수 있다.

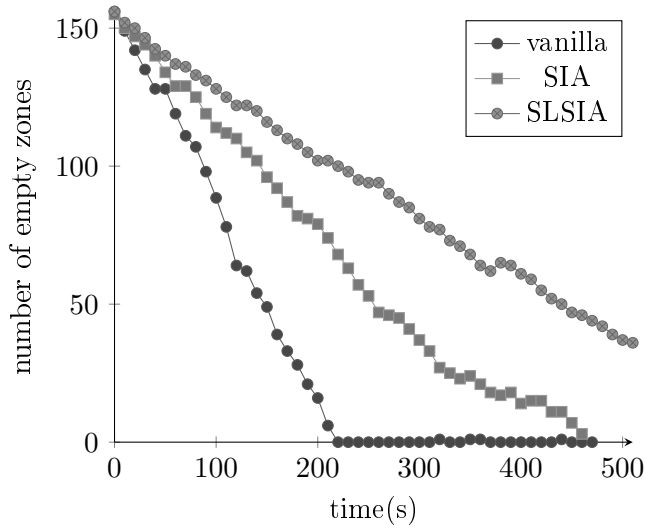
다만, SLSIA만의 두드러지는 한계점도 있다. 비슷한 시간에 끝난 다른 두 정책과 달리, 동일한 워크로드를 수행하는데 10초가량 더 오랜 시간이 소요되었다. 어쩌서 이러한 현상이 발생하였는지는 좀 더 분석이 필요할 것이다.



(a) (Accumulated) Compaction failure



(b) free space ratio



(c) number of empty zones

그림 5.1: 각 정책별 실험 결과

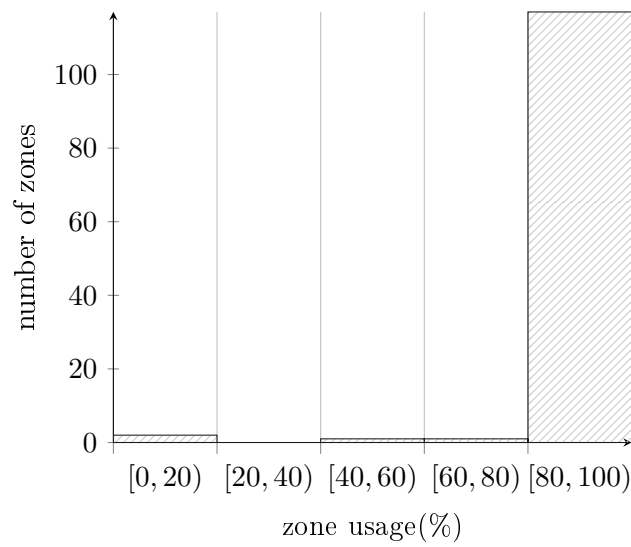


그림 5.2: 워크로드 종료 시점의 사용량 별 zone 분포
(empty zone은 집계하지 않음)

제6장 결론 및 향후 연구 방향

BlobDB와 ZenFS 간에 발생하는 다양한 문제를 분석하고 해결하였다. 아울러 추가적인 실험을 통해 성능 향상을 실증하였다. 반복적인 Compaction failure 문제와 space utilization 문제를 해결하였으며, 비교적 간단한 개선임에도 성능 개선이 상당함을 확인할 수 있었다.

향후 연구 방향은 다음과 같다. 먼저, 동일 조건의 벤치마크인데도 SLSIA 정책 하에서 유독 소요시간이 늘어지는 현상을 규명할 것이다. 워크로드의 크기와 상관 없이 항상 10초 가량 늘어지는 것이면 큰 문제는 아니겠지만, 만약 워크로드의 크기에 비례해서 늘어난다면 개선이 필요할 것이다.

또, 현행 연구는 db_bench만으로 성능 평가를 진행하였는데, db_bench의 워크로드만으로는 실제 키-밸류 스토어의 성능을 정확히 평가하기 힘들다. tail latency나 throughput 등 다양한 지표를 측정하지 못한 것도 아쉬운 부분이다. YCSB를 활용하거나, 아니면 YCSB도 그리 현실적이지 않다는 지적이 있으니만큼 [12], 다른 현실적인 workload generator(mixgraph 등)로 평가할 계획이다.

더 나아가서는, BlobDB단에서 개선할 부분을 더 찾는 것도 고려해 볼 수 있다. value separation의 효율은 Wiskey가 실증하였을지언정 BlobDB의 구현은 Wiskey와 상당히 다르기 때문에, 비교·분석하여 보면서 트레이드 오프를 보면 관찰을 것이다. 특히 BlobDB에서는 space amplification이 상당히 많이 일어나는데, 실제 운용시에는 공간 부족이 병목이 되는 상황 또한 흔하므로 [8], Wiskey의 전략을 참고하여 공간 효율을 높이는 방향을 고안해 볼 수도 있을 것이라 본다.

제7장 구성원 별 역할 및 개발 일정

7.1 연구일정

표 7.1에 나타난 연구 일정표는 본 연구의 전체적인 진행 과정을 월별로 나타낸 것이다. 시행착오가 꽤 있던만큼, 실제 연구가 일정표대로 선형적으로 이뤄지진 않았다. 그저 대략적인 흐름을 나타낸 것이며, 마찬가지로 업무 분장도 썩 깔끔하게 되지는 않았다.

표 7.1: 개발 일정표

연구구분	세부항목	5	6	7	8	9
연구 설계	주제 선정	○				
	선행연구 조사	○	○	○		
사전 준비	환경 구축	○	○			
	BlobDB, ZenFS 코드 스터디		○	○	○	
	추가적인 메트릭 수집 구현		○	○		
	버그 수정		○	○		
개선방안 도출	문제점 식별 및 개선 방향 탐색		○	○	○	
개선점 구현	ZenFS Zone 할당 정책 수정				○	○
	수정한 정책 디버깅				○	○
성능 실험 및 평가	성능 실험 및 평가					○

7.2 구성원 역할 분담

구성원별 역할 분담은 표 7.2와 같다.

표 7.2: 역할 분담

학번	성명	역할
202055524	김의현	BlobDB 분석 성능 개선을 위한 논문 스터디 주관 성능 평가 및 결과 분석
202055532	나도운	BlobDB 분석 BlobDB하에서 ZenFS 성능 분석 성능 평가 및 결과 분석
202055545	박정민	BlobDB 분석 BlobDB GC 개선점 탐색 성능 평가 및 결과 분석

제8장 산업체 멘토 자문의견서 대응 내역

8.1 Lifetime Hint 관련

알고리즘 8.1: (단순화한) BlobDB의 Lifetime Hint 계산

입력: 현재 Compaction/Flush의 output level
출력: lifetime hint
WLTH_MEDIUM = 3
WLTH_EXTREME = 5
if $0 \leq \text{level} < 1$ **then**
 lifetime_hint $\leftarrow$ WLTH_MEDIUM
else if $1 \leq \text{level} < 3$ **then**
 lifetime_hint $\leftarrow$ WLTH_MEDIUM + (level - 1)
else
 lifetime_hint $\leftarrow$ WLTH_EXTREME

먼저, 알고리즘 8.1이 RocksDB의 어디에서 발췌한 것인지 불명확하다는 의견이 있었다. 해당 알고리즘은 db/version_set.cc의 CalculateSSTWriteHint 함수 중 leveled Compaction에 해당하는 부분만을, base_level이 1이라는 전제 하에 정리한 것이다. Leveled Compaction만을 고려하였으니만큼 sst file의 lifetime은 WLTH_SHORT이 될 수 없다.

그리고 base_level은 L0이 Compaction 될 level을 의미한다. RocksDB에는 상황에 따라 L0을 L1 외의 level로 Compaction하는 기능이 있고 [13], 해당 기능은 base_level을 조정하는 식으로 동작하기 때문에(version_set.cc의 CalculateBaseBytes 함수 참조), 정확성을 위해서는 해당 동작 또한 기술하는 것이 맞을 것이다. 그러나 기본적으로 높은 레벨일수록 긴 lifetime hint가 부여된다는 기조는 동일하다. 따라서 지난 보고서에서는 base_level이 1인 상황을 가정하고 알고리즘 8.1과 같이 단순화하였다.

그러나 이번 보고서에서는 해당 알고리즘을 빼기로 하였다. 과한 생략으로 독자의 혼란을 유발하였고, 내용 전개 상 필수적인 내용도 아니기 때문이다. 대신 level이 높을수록 긴 lifetime hint를 부여한다는 서술로 갈음하였다.

또한 BlobDB 사용 시 Leveled Compaction 사용을 권장하고, BlobDB의 특정 기능은 Leveled Compaction 상에서만 동작한다 [9]. 따라서 Leveled Compaction만을 다루었는데,

지난번 중간보고서에서 해당 내용이 누락되어 독자 입장에서 이해할 수 없었으리라 생각한다. 따라서 2.4.2 절에 해당 언급을 추가하여 통해 이 부분을 보완하였다.

그리고 blob file의 lifetime hint 관련 설명이 모순이라는 지적이 있었다. 중간보고서의 설명이 모호하여 발생한 문제라고 생각하여, 3.2.1 절에서 더 명확하게 설명하였다. 다시 말하면, 어떤 Compaction/Flush의 결과로 생성된 모든 sst file과 blob file의 lifetime hint는 같다.

8.2 ZenFS의 파라미터 튜닝 관련

중간 보고서에서 언급한, ZenFS의 적정 parameter를 찾는 스크립트는 ZenFS repo¹에 있다. 해당 스크립트에 복잡한 로직은 존재하지 않으며, git blame으로 변경 내역을 확인해본 결과, 초기 커밋 이후로 아무런 수정이 이뤄지지 않았다.

아울러 RocksDB와 ZenFS를 다루는 다른 연구에도 zone의 크기가 크면(즉, sst file이 zone 보다 작으면) ZenFS의 효율이 떨어진다는 언급이 있다 [14]. 다만 ZenFS에 GC가 도입되기 이전 시점의 연구이니만큼, GC 이후로 상황이 얼마나 달라졌는지는 불명확하다.

8.3 WAF 측정 관련

$$\text{RocksDB WAF} = \frac{(\text{write during Compaction}) + (\text{write during Flush})}{(\text{write during Flush})} \quad (8.1)$$

$$\text{ZenFS WAF} = \frac{(\text{user written data}) + (\text{gc written data})}{(\text{user written data})} \quad (8.2)$$

RocksDB, ZenFS의 WAF를 어떻게 측정한 것인지 불분명하다는 지적이 있었다. RocksDB의 WAF 측정은 식 8.1과 같고, ZenFS는 식 8.2와 같으며, ZenFS의 경우 3.1.2절에서 언급하였다. 중간보고서에 기재된 BlobDB의 write amplification은 오류였음을 밝히는 바이다 (분자를 구하는 과정에서 Flush가 두 번 더해짐, 즉 $\text{Compaction} + \text{Flush} + \text{Flush}$ 로 계산됨). 아울러, RocksDB/BlobDB의 write amplification은 그리 중요한 내용이 아닌 만큼 이번 보고서에 포함되지 않았다.

8.4 ‘문제 정의’, ‘과제 목표’ 관련

이전까지의 목표는 BlobDB 측에서 lifetime hint를 개선하고, 그에 따라 ZenFS도 최적화하는 방향의 연구를 구상하였다. 그러나 중간보고서 시점까지도 ZenFS와 BlobDB간의 연동에

¹tests/get_good_db_bench_params_for_zenfs.sh

문제가 너무 많았고, 벤치마크를 완료할 수 있는 설정값을 찾는 것조차 힘들었다. 따라서 과제 목표를 본 보고서와 같이 수정하였다.

참고문헌

- [1] *RocksDB-Overview*. Accessed: May 13, 2025. [Online]. Available: <https://github.com/facebook/rocksdb/wiki/RocksDB-Overview>.
- [2] *leveldb*. Accessed: Sep. 11, 2025. [Online]. Available: <https://github.com/google/leveldb>.
- [3] L. Lu, T. S. Pillai, H. Gopalakrishnan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, “WiscKey: Separating Keys from Values in SSD-Conscious Storage,” *ACM Trans. Storage*, vol. 13, no. 1, 5:1–5:28, 2017. DOI: 10.1145/3033273.
- [4] *Badger*. Accessed: Sep. 3, 2025. [Online]. Available: <https://github.com/hypermodeinc/badger>.
- [5] L. Tamasi, *Integrated BlobDB*, May 2021. Accessed: May 13, 2025. [Online]. Available: <https://rocksdb.org/blog/2021/05/26/integrated-blob-db.html>.
- [6] M. Bjørling et al., “ZNS: Avoiding the block interface tax for flash-based SSDs,” in *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, USENIX Association, 2021, pp. 689–703.
- [7] P. O’Neil, E. Cheng, D. Gawlick, and E. O’Neil, “The log-structured merge-tree (LSM-tree),” *Acta Inf.*, vol. 33, no. 4, pp. 351–385, Jun. 1996.
- [8] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, “RocksDB: Evolution of Development Priorities in a Key-value Store Serving Large-scale Applications,” *ACM Trans. Storage*, vol. 17, no. 4, 26:1–26:32, Oct. 2021. DOI: 10.1145/3483840.
- [9] *BlobDB*. Accessed: Sep. 3, 2025. [Online]. Available: <https://github.com/facebook/rocksdb/wiki/BlobDB>.
- [10] H. Li, M. Hao, M. H. Tong, S. Sundararaman, M. Bjørling, and H. S. Gunawi, “The CASE of FEMU: Cheap, Accurate, Scalable and Extensible Flash Emulator,” in

- Proceedings of the 16th USENIX Conference on File and Storage Technologies*, 2018, pp. 83–90.
- [11] I. Song, M. Oh, B. S. J. Kim, S. Yoo, J. Lee, and J. Choi, “ConfZNS : A Novel Emulator for Exploring Design Space of ZNS SSDs,” *Proceedings of the 16th ACM International Conference on Systems and Storage*, pp. 71–82, Jun. 2023. DOI: 10.1145/3579370.3594772.
- [12] Z. Cao, S. Dong, S. Vemuri, and D. H. C. Du, “Characterizing, Modeling, and Benchmarking RocksDB Key-Value Workloads at Facebook,” in *Proceedings of the 18th USENIX Conference on File and Storage Technologies*, 2020, pp. 209–223.
- [13] *Leveled-Compaction*. Accessed: Sep. 18, 2025. [Online]. Available: <https://github.com/facebook/rocksdb/wiki/Leveled-Compaction>.
- [14] 이정은, “작은 존 ZNS SSD에 맞는 RocksDB와 ZenFS 분석 및 최적화,” 석사학위논문, 서울대학교 대학원, 2022.